Method	Structures it Touches	Big - O	One Line Reason
Add song	DLL and HashMap	O(1)	We already know where the end is, so we just stick the new song on and update two map entries
Find song	HashMap,ArrayDeque, merge	O(1)	The hash map jumps straight to the song using its id, like looking up a word by page number instead of reading the whole dictionary.
play	HashMap,ArrayDeque	O(1)	One map lookup to find the song, one quick push onto the history stack, one count update
Queue later	HashMap,ArrayDeque	O(1)	A map check to make sure the song exists, then drop it at the back of the line
Queue next	HashMap,ArrayDeque	O(1)	Same check, then put it at the front of the line
next	HashMap,ArrayDeque,DLL	O(1)	Normally: pop one song, push one to history, step to the next node
back	HashMap,ArrayDeque,DLL	O(1)	Normally: pop one song from history, push the old one to the front of up next
Remove song	HashMap and DLL	O(1)	The map finds the

			node instantly, and because the node knows both its left and right neighbor, we just connect them to each other and the node is gone
Show Playlist	DLL	O(N)	To list every title we have to visit every node one by one from the start,

I used a hash map for the library because its whole job is to find a song fast when someone gives me its id and hash map does that in one step it jumps straight to the answer instead of looking through every song. I made it store the playlist node not just the song so once I find the node I also instantly know its left and right neighbors in the playlist. That means when I play the next song or remove a song I don't have to search the playlist at all. I just follow the pointers I already have. If I had used a plain array instead, every find song by id would mean checking songs one by one until I found it, and removing a song would mean hunting for where it sits before I could take it out both get slower as the playlist grows. The built in hash map costs a little extra memory, but it saves me from writing my own lookup from scratch and the speed win is worth it.

I used a deque for history because the back button always takes the most recent song. A deque can add and remove at its front in one step. So pressing back is instant no matter how many songs are in history. When I loop over a deque used like this it already gives me the newest song first. If I had used an array pressing back would mean pushing everything else over by one spot every time. Which gets slower the longer the history grows and I have to keep track of where the top is and handle the array getting full myself. Reusing the built in deque means Java handles all that bookkeeping for me. The cost is a little unused space inside it but the operations stay fast.